

REDES DE COMPUTADORAS

CURSO 2026

GRUPO 19

Informe - Obligatorio 01

Autores:

Luca SCABONI

Catalina SEGUI

Olivia ZORRILLA

Supervisor:

Eduardo GRAMPÍN

11 de septiembre de 2026

Índice

1. Parte 1	2
1.1. Descripción y Comparación de Tcpdump y Wireshark	2
1.2. Análisis de Sesiones TCP en el Loopback	3
1.3. Captura y análisis de tráfico HTTP y HTTPS	4
1.4. Reflexión	6
2. Parte 2	7
2.1. Componentes del Sistema	7
2.2. Interfaz y Cálculo del Broadcast	8
2.3. Modelo de Hilos	8
2.4. Identificación de Agentes	11
2.5. Locks	11
2.6. Coordinación de Consultas de Procesos	12
2.7. Alertas y Archivos de Log	13
2.8. Errores y Cierre de Conexiones	13
2.9. Pruebas Realizadas	14
3. Anexo	16
3.1. Referencias	16
3.2. Declaración de Uso de IA	16

1. Parte 1

1.1. Descripción y Comparación de Tcpdump y Wireshark

Tcpdump

`tcpdump` es una herramienta de código abierto utilizada para capturar y analizar tráfico de red desde la línea de comandos. Su funcionamiento se basa en la biblioteca `libpcap`, que permite acceder a los paquetes que circulan por las interfaces de red de un equipo [1].

Se encuentra disponible principalmente para sistemas operativos de tipo Unix, como Linux y macOS. En Windows puede utilizarse mediante WSL o mediante adaptaciones como WinDump.

La herramienta permite capturar paquetes en vivo, seleccionar la interfaz de red que se desea observar y guardar las capturas en archivos de formato `pcap` para analizarlas posteriormente. También permite aplicar filtros según criterios como protocolo, dirección IP, puerto, red, origen o destino. Estos filtros utilizan la sintaxis BPF proporcionada por la biblioteca `libpcap` [1, 2].

Wireshark

Wireshark es una herramienta gratuita y de código abierto destinada a la captura y el análisis gráfico del tráfico de red. Está disponible para Windows, macOS, Linux y otros sistemas de tipo Unix [3].

Permite capturar paquetes en vivo, elegir las interfaces de red y guardar las capturas para examinarlas posteriormente. Su interfaz muestra la lista de paquetes, los campos de los protocolos que componen cada uno y su contenido en formato hexadecimal y ASCII.

Wireshark dispone de filtros de captura, que determinan qué paquetes se almacenan, y filtros de visualización, que permiten seleccionar qué paquetes se muestran después de realizada la captura. Además, ofrece funciones para estudiar conversaciones, consultar estadísticas y analizar detalladamente una gran cantidad de protocolos [4].

Comparación

`tcpdump` y Wireshark permiten capturar tráfico en vivo, seleccionar interfaces, aplicar filtros y guardar los paquetes para analizarlos posteriormente. Sin embargo, no son completamente equivalentes.

`tcpdump` es una herramienta liviana y orientada al trabajo desde la terminal, por lo que resulta conveniente para capturas rápidas, servidores y equipos remotos. Wireshark, en cambio, proporciona una interfaz gráfica y herramientas más completas para inspeccionar e interpretar los paquetes.

Sin embargo, ambas herramientas son interoperables, pues los archivos `pcap` generados por `tcpdump` pueden abrirse y analizarse con Wireshark. Por lo tanto, pueden utilizarse de manera complementaria.

1.2. Análisis de Sesiones TCP en el Loopback

Los mensajes enviados en una sesión Telnet no pueden observarse desde la otra sesión, aunque ambas estén conectadas a la misma dirección IP y al mismo puerto del servidor. Cada sesión corresponde a una conexión TCP independiente y el sistema operativo entrega los datos únicamente al socket asociado con dicha conexión. Los paquetes de ambas sesiones sí aparecen mezclados cronológicamente en la captura, ya que Wireshark registra todo el tráfico que circula por la interfaz seleccionada.

Una conexión TCP se identifica mediante la combinación de dirección IP y puerto de origen, dirección IP y puerto de destino, junto con el protocolo de transporte. En este caso, como todo el tráfico circula por la interfaz de loopback, las direcciones de IP de origen y destino son siempre 127.0.0.1. La diferencia entre las sesiones se encuentra en el puerto utilizado por cada cliente.

Como se observa en la captura `servidor.pcap`, se establecieron tres conexiones TCP diferentes:

- Primera sesión: 127.0.0.1:60696 ↔ 127.0.0.1:2026
- Segunda sesión: 127.0.0.1:60708 ↔ 127.0.0.1:2026
- Tercera sesión: 127.0.0.1:60716 ↔ 127.0.0.1:2026

El puerto 2026 corresponde al servidor, mientras que los puertos 60696, 60708 y 60716 fueron asignados a los clientes. Por lo tanto, para determinar a qué sesión pertenece un mensaje, se deben observar los puertos de origen y destino.

Wireshark también asigna un identificador `tcp.stream` diferente a cada conexión TCP. Este campo permite filtrar todos los paquetes pertenecientes a una misma sesión, independientemente del sentido en el que circulen. De este modo, aunque los paquetes de las tres conexiones aparezcan intercalados en la captura, es posible separarlos e identificar a qué sesión pertenece cada mensaje.

Por ejemplo, para visualizar únicamente los paquetes de la primera conexión se utilizó el siguiente filtro de visualización:

```
tcp.stream eq 0
```

Luego de aplicar el filtro solamente permanecen los paquetes intercambiados entre el cliente 127.0.0.1:60696 y el servidor 127.0.0.1:2026.

Para comprobar también el contenido intercambiado, se utilizó la opción *Follow TCP Stream* de Wireshark sobre cada una de las conexiones. Esta herramienta reconstruye los datos intercambiados dentro de una única conexión TCP y representa con colores diferentes los mensajes enviados por el cliente y las respuestas del servidor.

Realizando lo anterior, se puede observar que cada flujo contiene exclusivamente las solicitudes realizadas desde uno de los clientes y las respuestas enviadas por el servidor a ese mismo cliente. En las tres sesiones se enviaron los mismos mensajes, como `par 1`, `par 2`, `par 3` e `impar 4`, sin embargo, los números devueltos por el servidor son diferentes en cada conexión y no aparecen en los otros flujos.

Esto confirma que, aunque las tres sesiones se comuniquen con 127.0.0.1:2026 y sus paquetes aparezcan intercalados en la captura, los datos de una sesión no son recibidos por los demás.

1.3. Captura y análisis de tráfico HTTP y HTTPS

En la captura realizada al acceder a la URL `https://www.fing.edu.uy`, se filtraron los paquetes correspondientes a la comunicación entre el host cliente y

el servidor con dirección IP 164.73.32.20 y puerto 443. Se utilizó el comando `ping` para obtener la dirección IP del servidor asociada al nombre de dominio `www.fing.edu.uy`. A partir de la captura, y como vimos anteriormente en el punto anterior, se observa que el cliente establece distintas conexiones TCP con el servidor utilizando distintos puertos de origen.

Si bien los mensajes en el protocolo HTTP son compuestos por texto, en este caso, el acceso al sitio fue realizado mediante HTTPS, donde se cifra la información intercambiada entre el cliente y el servidor. En la captura, se observa el uso del protocolo TLSv1.3, el cual es una versión modificada del protocolo SSL, que utiliza técnicas de criptografía para mejorar los servicios de seguridad de TCP, incluyendo la confidencialidad y la integridad de los datos [5]. Por este motivo, aunque los mensajes HTTP existen, su contenido no puede visualizarse como texto plano en Wireshark.

Para acceder al sitio de NeverSSL se utiliza HTTP sin cifrado, por lo que en Wireshark es posible observar directamente los mensajes HTTP intercambiados, incluyendo las solicitudes GET, las respuestas del servidor y sus encabezados. Esto difiere del acceso al sitio web de Fing, realizado mediante HTTPS, donde el contenido HTTP se encuentra cifrado mediante TLS y, por lo tanto, en Wireshark se visualiza principalmente como *Application Data*.

El encabezado **Content-Type** indica el tipo de contenido incluido en el cuerpo de la respuesta HTTP y, opcionalmente, la codificación de caracteres utilizada [5]. Por otra parte, el encabezado **Content-Length** indica el tamaño del cuerpo de la respuesta expresada en bytes [5].

Consideremos el paquete 336 de la captura incluida en el archivo `http_web.pcap`. En la respuesta HTTP se observa el encabezado **Content-Type: text/html; charset=iso-8859-1**, que indica que el recurso enviado corresponde a una página web escrita en HTML y utiliza la codificación de caracteres ISO-8859-1. Por otro lado, el encabezado **Content-Length: 298** indica que el cuerpo de la respuesta tiene una longitud de 298 bytes.

Un aspecto observado, que consideramos interesante y relevante a incluir, fue el uso de diferentes versiones del protocolo IP. El dominio `neverssl.com` dispone tanto de una dirección IPv4 como de una dirección IPv6. Al realizar dos capturas,

una desde macOS con Safari, la conexión se estableció mediante la dirección IPv4 34.223.124.45, que coincidía con la dirección obtenida mediante `ping`. En cambio, en la captura realizada desde Windows 11 con Chrome, el navegador se comunicó con el sitio mediante la dirección IPv6 2600:1f13:....

Esta diferencia no implica necesariamente una diferencia en el manejo de direcciones por parte de los sistemas operativos o navegadores. Cuando un dominio ofrece ambas posibilidades, la elección puede depender de la configuración de la red, la disponibilidad de conectividad IPv6 y del tiempo de respuesta de cada alternativa. Por esta razón, dos equipos que acceden al mismo dominio pueden comunicarse con el mismo servicio utilizando familias de direcciones distintas.

Además, que `ping` muestre una dirección IPv4 no significa que el dominio disponga únicamente de esa dirección. La herramienta selecciona una de las direcciones obtenidas mediante DNS, mientras que el navegador puede utilizar otra alternativa al establecer la conexión.

1.4. Reflexión

El hecho de que `ping` falle no permite concluir por sí solo que el sitio web se encuentre fuera de servicio. Esta herramienta envía mensajes ICMP Echo Request y espera recibir mensajes Echo Reply desde el destino. Si no obtiene respuesta, puede deberse a que el servidor no está disponible, pero también a que este o algún equipo intermedio no responde a ese tipo de mensajes. Por lo tanto, el resultado solamente indica que no se recibieron las respuestas esperadas por `ping`.

Por otra parte, `traceroute` intenta identificar los routers atravesados por los paquetes hasta llegar al destino. Para hacerlo, envía paquetes con valores de TTL progresivamente mayores y utiliza las respuestas de equipos intermedios para reconstruir el recorrido. Si deja de mostrar resultados al llegar a la red del proveedor de hosting, no necesariamente significa que los paquetes se detengan allí. También es posible que los siguientes routers o el propio servidor no respondan a las pruebas de `traceroute`, aunque continúen reenviando otro tipo de tráfico.

A partir de estos resultados podrían plantearse distintas hipótesis: que el servidor se encuentre fuera de servicio es una de ellas, sin embargo, pensar que existe un problema en algún punto del recorrido o que simplemente no se estén respondiendo

los mensajes utilizados por `ping` y `traceroute` también es una opción válida. Antes de obtener una conclusión, convendría intentar acceder al sitio desde el navegador, repetir las pruebas desde otro equipo o red y realizar una captura con Wireshark o `tcpdump`. En la captura podría observarse si el cliente intenta establecer una conexión TCP con el servidor y si recibe alguna respuesta. De esta manera, se podría distinguir entre la falta de respuesta a las herramientas de diagnóstico y una posible indisponibilidad del sitio web.

2. Parte 2

Esta sección describe la arquitectura de la 2da parte del laboratorio, las decisiones tomadas durante la implementación y la razón de cada una. El sistema está formado por un servidor, clientes comunes que reportan información del equipo en el que se ejecutan y clientes administradores que consultan dicha información.

2.1. Componentes del Sistema

El sistema se divide en cuatro archivos principales:

- `servidor.py`: Responde solicitudes de descubrimiento mediante UDP, acepta conexiones TCP, mantiene el estado de los agentes y atiende consultas de administradores.
- `clientes_comun.py`: Registra un cliente común, mide CPU y memoria, informa alertas y responde consultas sobre procesos.
- `clientes_admin.py`: Presenta una consola interactiva que permite elegir entre las funcionalidades administrativas, es decir, listar agentes y consultar métricas o procesos.
- `utilidades_red.py`: Contiene la lógica común de sockets, descubrimiento y envío y recepción de mensajes TCP.

Contraseña y Umbrales

La clave utilizada para autenticar a los clientes se definió como una constante `CLAVE`. Igualmente, se definieron como constantes los umbrales de CPU y memoria.

2.2. Interfaz y Cálculo del Broadcast

Una máquina puede tener varias interfaces: Ethernet, Wi-Fi, WSL, VPN, loopback, etc. Cada interfaz puede pertenecer a una subred diferente. Por ese motivo, el programa muestra las interfaces IPv4 disponibles mediante el uso de la librería `psutil` y permite que el usuario elija cuál usar.

La selección devuelve tres datos:

```
nombre de interfaz, dirección IPv4, máscara de subred
```

Con la dirección IPv4 y la máscara seleccionadas se calcula el broadcast correspondiente a esa subred, en lugar de utilizar directamente la dirección `255.255.255.255`.

Para realizar el cálculo utilizamos `ipaddress.IPv4Network`. Esta clase recibe una dirección IPv4 junto con su máscara y permite obtener información sobre la red a la que pertenece, como su dirección de red y su dirección de broadcast.

Por defecto, `IPv4Network` exige que la dirección proporcionada sea la dirección inicial de la red (Ej.: IP de la Interfaz: `192.168.1.2` → Dirección Inicial de la Red: `192.168.1.0`).

Como nosotros le proporcionamos la dirección particular de la interfaz y no la dirección de red, utilizamos `strict=False`. Esta opción permite que la clase acepte una dirección de host y determine automáticamente la red correspondiente a partir de su máscara.

Finalmente, mediante el atributo `broadcast_address`, obtenemos la dirección de broadcast de la red calculada.

2.3. Modelo de Hilos

Servidor

El servidor debe ser capaz de aceptar nuevos clientes mediante UDP, aceptar nuevas conexiones TCP y también aceptar a los clientes ya conectados. Si todas las conexiones se atendieran de forma secuencial, mientras el servidor procesa los mensajes de un cliente no podría aceptar ni atender a otros clientes.

Por esta razón, se decidió utilizar una arquitectura basada en hilos:

1. Hilo principal que espera y acepta nuevas conexiones TCP..
2. Un hilo dedicado al descubrimiento UDP, se encarga de recibir los mensajes DISCOVER.
3. Un hilo por cada conexión TCP aceptada, procesa los mensajes de los clientes.

Cliente Común

El cliente común posee dos flujos de ejecución:

1. Recibe `GET_PROC` y responde `PROC`.
2. Monitorea CPU y memoria.

El hilo de monitoreo utiliza `psutil.cpu_percent(interval=1)`, por lo que realiza mediciones cada 1 segundo. En cada vuelta también consulta el porcentaje de memoria y compara ambos valores con los umbrales enviados por el servidor.

De este modo, las alertas se comprueban cada segundo. Esto permite detectar que se superó el umbral definido, independientemente de los quince segundos que separan los envíos de métricas.

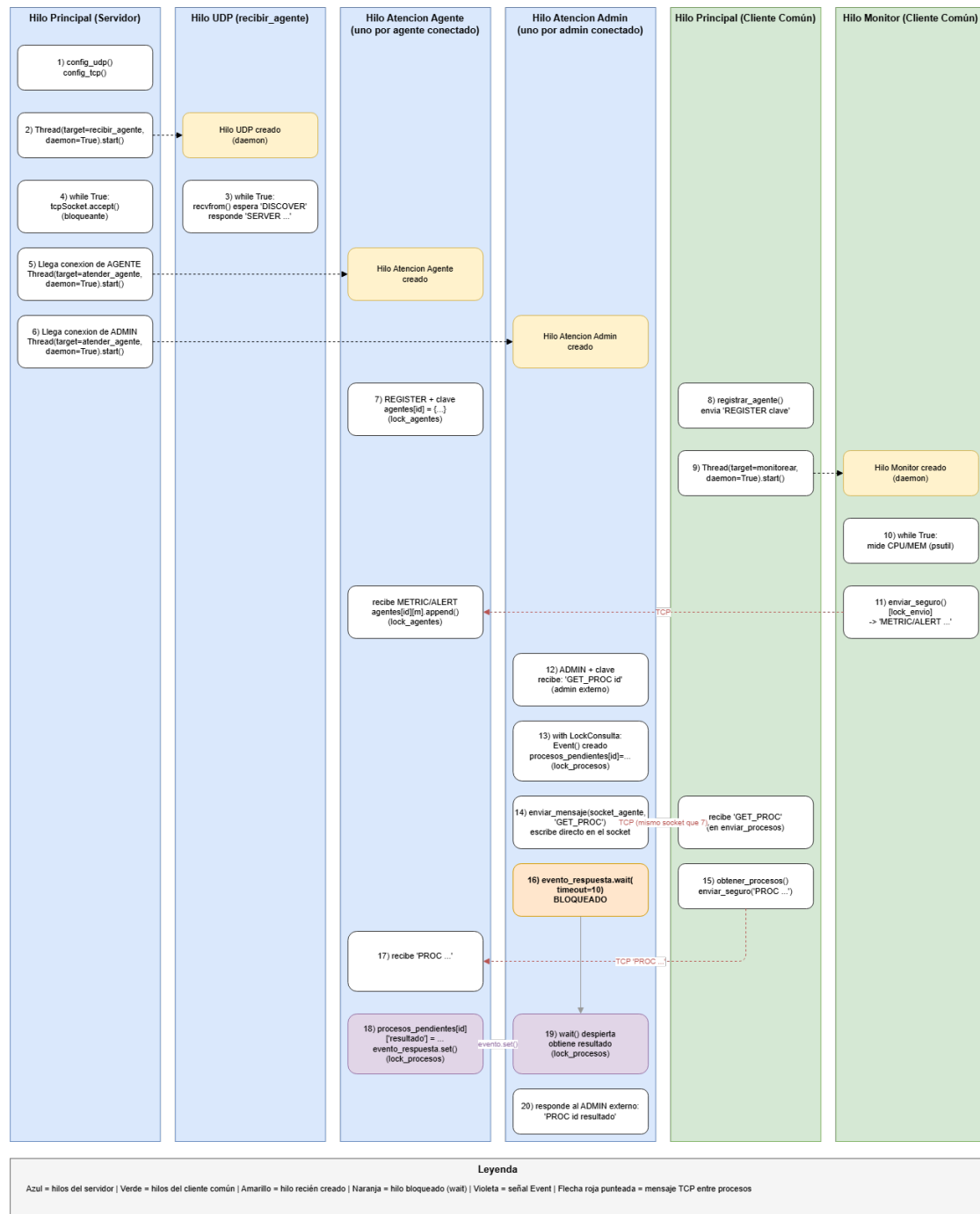
Administrador

El cliente administrador posee un único hilo de ejecución. Este hilo espera que el usuario ingrese un comando por consola, envía la solicitud correspondiente al servidor y espera hasta recibir su respuesta.

Diagrama de Hilos

Observar la siguiente figura que representa un diagrama de hilos del sistema y como estos se comunican. En particular, se puede observar el intercambio de mensajes entre hilos de servidor y cliente común para lograr obtener los procesos del cliente y enviárselos al administrador.

Diagrama de Hilos



2.4. Identificación de Agentes

Cada agente común es identificado con la dirección de su conexión TCP:

`<IP : puerto_origen>`.

El servidor mantiene una estructura llamada **agentes** que se indexa por el identificador del agente y para cada uno almacena su dirección, socket TCP, las últimas 10 métricas de CPU y MEM enviadas por el agente y un lock para la consulta de procesos.

El socket se conserva porque el servidor debe poder enviar `GET_PROC` al agente cuando sea solicitado por un administrador. Y el lock, llamado `lock_consulta`, serializa las consultas de procesos dirigidas a ese agente (ver Sección 2.6).

2.5. Locks

Lock de Envío del Cliente Común

El hilo de monitoreo y el hilo principal pueden enviar mensajes simultáneamente por el mismo socket TCP:

- El monitor envía `METRIC` y `ALERT`.
- El hilo principal responde `PROC`.

Como `enviar_mensaje()` puede ejecutar varias llamadas a `send()`, dos hilos podrían intercalar partes de sus mensajes. El lock `lock_envio` garantiza que un mensaje completo termine de enviarse antes de comenzar el siguiente.

Lock del Diccionario de Agentes

La estructura **agentes** es compartida por todos los hilos TCP. Por esta razón se decidió utilizar un `lock_agentes` que garantice la mutua exclusión entre hilos sobre dicha estructura, tanto para leerla o modificarla, con el fin de evitar condiciones de carrera.

Por ejemplo, un admin podría consultar por los procesos de un cliente común al mismo tiempo que el hilo correspondiente a ese cliente lo elimina de la estructura

debido a una desconexión, el uso del lock permite que estas operaciones sobre agentes no se realicen en simultáneo.

2.6. Coordinación de Consultas de Procesos

La consulta de procesos involucra dos conexiones y dos hilos distintos del servidor:

- El hilo que atiende al admin recibe `GET_PROC <id>`.
- El hilo que atiende al cliente común recibe posteriormente `PROC <lista>`.

El servidor utiliza la estructura `procesos_pendientes` para relacionarlos:

```
procesos_pendientes[id_agente] = {  
    "evento": evento,  
    "resultado": None  
}
```

Uso de Event

`threading.Event` representa una señal entre hilos.

Inicialmente, se crea apagado. El hilo del administrador espera mediante `wait(timeout=10)`, mientras que el hilo del agente se encarga de guardar el resultado en `procesos_pendientes` y ejecuta `set()`, que despierta al hilo del admin.

El timeout evita que el admin espere indefinidamente si el agente no responde.

Lock de Procesos Pendientes

La estructura `procesos_pendientes` es utilizada por dos hilos diferentes. El hilo del administrador agrega y elimina consultas, mientras que el hilo del cliente común busca la consulta correspondiente y guarda en ella el resultado recibido.

Por este motivo se utiliza `lock_procesos`, que garantiza que solamente un hilo por vez pueda consultar o modificar esta estructura.

Es importante liberar `lock_procesos` antes de ejecutar `wait()`. Si el hilo del administrador mantuviera el lock mientras espera, el hilo del cliente común no podría

adquirirlo para guardar la respuesta y activar el evento. Ambos hilos quedarían bloqueados.

Serialización por Agente

Dos administradores pueden consultar simultáneamente los procesos de un mismo cliente común. Como el protocolo `PROC` no incluye un identificador de solicitud, dos respuestas concurrentes no podrían asociarse fácilmente con sus pedidos.

Cada agente tiene un `lock_consulta`. El primer admin lo adquiere y realiza toda la consulta. El segundo espera ese mismo lock y comienza su propia consulta después de que la primera termina.

El lock es individual por agente, de modo que no se bloquean consultas destinadas a agentes diferentes.

El envío final de `PROC` al admin se realiza después de liberar `lock_consulta`. Así la siguiente consulta puede comenzar sin esperar a que finalice el envío sobre otra conexión.

2.7. Alertas y Archivos de Log

Cuando el servidor recibe una alerta válida, la muestra y la registra con `logging.warning()` en `alertas.log`. Este log incluye fecha, hora, identificador del agente, métrica y valor.

2.8. Errores y Cierre de Conexiones

La aplicación distingue entre los errores del protocolo y los errores producidos por la pérdida de una conexión TCP.

Errores del Protocolo

El mensaje `ERROR` se utiliza cuando un extremo recibe un mensaje que no puede procesar. Esto puede ocurrir porque el comando no es reconocido, porque no tiene la cantidad esperada de parámetros o porque alguno de sus valores no es válido. Por ejemplo, el servidor responde `ERROR` si recibe una métrica distinta de `CPU` o `MEM`.

Cuando un extremo recibe **ERROR**, solamente informa lo ocurrido y no responde con otro mensaje **ERROR**. Esta decisión evita que ambos extremos entren en un ciclo en el que cada mensaje de error genere otro también de error.

Siempre que es posible, el administrador valida los comandos ingresados antes de enviarlos. Por ejemplo, comprueba que el número de agente sea válido y que la métrica solicitada sea **CPU** o **MEM**. De esta forma, los errores simples de entrada se resuelven localmente y no generan tráfico innecesario.

Cierre Ordenado

El mensaje **END** representa una solicitud de cierre ordenado. El cliente lo envía cuando el administrador escribe **END** o interrumpe la ejecución mediante el teclado.

Cuando el servidor recibe este mensaje, ejecuta **break** y abandona el bucle que atendía la conexión. Abandonar el bucle no evita la ejecución del bloque **finally**, donde se liberan los recursos asociados a la conexión.

Desconexión Inesperada

Una conexión también puede finalizar sin que se intercambie **END**. Esto puede ocurrir, por ejemplo, si se apaga el servidor, se cierra uno de los clientes o se interrumpe la comunicación.

Las funciones **enviar_mensaje** y **recibir_mensaje** detectan este caso. Si **recv()** devuelve una secuencia de bytes vacía, significa que el otro extremo cerró la conexión. De forma similar, si **send()** informa que envió cero bytes, no es posible continuar con el envío. En ambos casos, se genera una excepción **ConnectionError**.

El servidor captura esta excepción para evitar mostrar un error técnico en la consola y permite que la ejecución continúe hacia el bloque **finally**. Los clientes también la capturan, muestran el mensaje **SE PERDIÓ LA CONEXIÓN CON EL SERVIDOR** y finalizan su ejecución de forma controlada.

2.9. Pruebas Realizadas

Las pruebas de la aplicación se realizaron en distintas etapas y utilizando diferentes entornos de ejecución.

Pruebas Individuales

Durante el desarrollo, cada integrante realizó pruebas de manera individual sobre los distintos intercambios definidos por el protocolo. Se verificaron el descubrimiento del servidor mediante UDP, el registro de clientes mediante TCP, el envío periódico de métricas y alertas, y las consultas realizadas desde el administrador.

Estas pruebas permitieron detectar errores durante la implementación y comprobar el funcionamiento básico de cada componente antes de realizar pruebas entre equipos diferentes.

Pruebas en la Red de la Facultad

Posteriormente, se realizó una prueba utilizando tres equipos Unix conectados a una misma red de la facultad. Cada equipo ejecutó uno de los roles de la aplicación.

El cliente común y el administrador seleccionaron sus respectivas interfaces de red y lograron descubrir al servidor mediante UDP. Luego establecieron sus conexiones TCP y se verificó el intercambio de información entre los tres componentes. El administrador pudo listar el agente conectado, consultar sus métricas y solicitar su lista de procesos. La prueba finalizó correctamente.

Prueba con Dispositivos Personales

Finalmente, se realizó una prueba utilizando una computadora con macOS, una computadora con Windows y un teléfono celular utilizado como punto de acceso Wi-Fi. Ambas computadoras se conectaron a la red creada por el teléfono y ejecutaron los distintos componentes de la aplicación.

Los programas seleccionaron las interfaces Wi-Fi correspondientes, calcularon el broadcast de la red y lograron realizar el descubrimiento y las conexiones necesarias. Una vez establecida la comunicación, las computadoras intercambiaron correctamente los mensajes y la información solicitada.

Esta prueba permitió comprobar el funcionamiento de la aplicación entre sistemas operativos diferentes y en una red distinta de la utilizada durante el desarrollo.

3. Anexo

3.1. Referencias

- [1] The Tcpdump Group. *tcpdump(1) manual page*. Disponible en: <https://www.tcpdump.org/manpages/tcpdump.1.html>.
- [2] The Tcpdump Group. *pcap-filter: packet filter syntax*. Disponible en: <https://www.tcpdump.org/manpages/pcap-filter.7.html>.
- [3] Wireshark Foundation. *Wireshark User's Guide: Introduction*. Disponible en: https://www.wireshark.org/docs/wsug_html_chunked/ChapterIntroduction.html.
- [4] Wireshark Foundation. *Wireshark User's Guide: Filtering while capturing*. Disponible en: https://www.wireshark.org/docs/wsug_html_chunked/ChapterCaptureFilterSection.html.
- [5] James F. Kurose y Keith W. Ross. *Redes de computadoras: Un enfoque descendente*. 7.a edición, Pearson Educación, Madrid, 2017.

3.2. Declaración de Uso de IA

Se utilizaron herramientas de Inteligencia Artificial como apoyo para consultas de sintaxis, identificación de librerías de Python, como `ipaddress` que fue utilizada para el cálculo de broadcast, y aclaración de conceptos puntuales de implementación, como el formateo de una lista de procesos mediante el uso `join()`. El diseño, desarrollo y validación de la solución fueron realizados por el equipo.